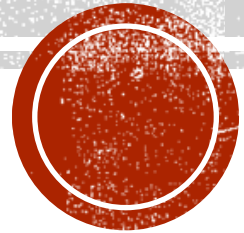
COS 201- COMPUTER PROGRAMMING II

MISS OYEWOLE C.B



PACKAGE

A Package can be defined as a grouping of related types(classes, interfaces). Java provides a large number of classes grouped into different packages based on their functionality.

The classes contained in the packages of other programs/applications can be reused. There are two types of packages in Java namely;

1. **Pre-defined Packages(built-in):** these packages consist of a large number of classes which are a part of Java API (Application Program Interface).
2. **User defined packages**

Pre-defined package include: java.lang, java.io, java.util, java.applet, java.awt, java.net, javax.swing and java.sql

- **Java.lang**: Contains language support classes (for e.g classes which defines primitive data types, math operations, etc.). This package is automatically imported.
- **Java.io**: Contains classes for supporting input / output operations.
- **Java.util**: Contains utility classes which implement data structures like Linked List, Hash Table, Dictionary, etc and support for Date / Time operations. This package is also called as **Collections.**

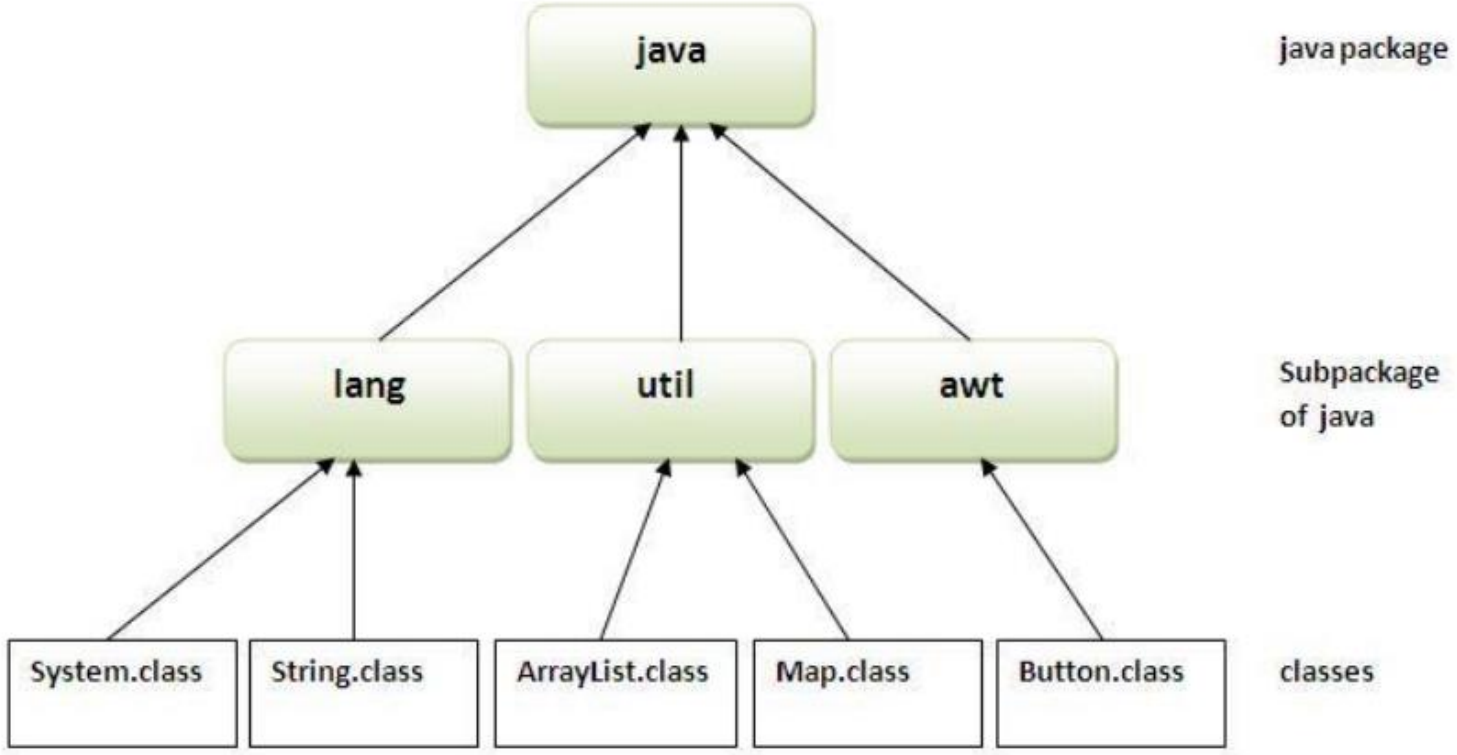


PRE-DEFINED PACKAGE- CONT'D

- **Java.applet**: Contains classes for creating Applets.
- **Java.awt**: Contains classes for implementing the components of graphical user interface (like buttons, menus, etc.).
- **Java.net**: Contains classes for supporting networking operations.
- **Javax.swing**: This package helps to develop GUI Applications. The 'x' in javax represents that it is an extended package which means it is a package developed from another package by adding new features to it. In fact, javax.swing is an extended package of java.awt.
- **Java.sql**: This package helps to connect to databases like Oracle/Sybase/Microsoft Access to perform different operations.



JAVA FOUNDATION PACKAGES



HOW TO ACCESS PACKAGE FROM ANOTHER PACKAGE

To use a class or a package from the Java API, the keyword ***import*** should be used. The import keyword is used to make the classes and interface of another package accessible to the current package.

There are three ways to access the package from outside the package;

- Using packagename*: To import a whole package, end the sentence with an asterisk sign (*). The following example will import all the classes and interfaces in the *java.util* package but not subpackages:

import. java.util. *;



ACCESSING CLASSES INSIDE A PACKAGE

- Import/calling a specific class by using packagename.classname: which means only declared class of this package will be accessible. E.g

Import java.util.Vector;

This imports only the vector class from the *java.util* package

- Using fully qualified name

If you use fully qualified name, then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.



USER-DEFINED PACKAGE

- These are packages defined by the user.
- User defined package is achieved by including a ***package*** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package.
- The package statement defines a name space in which classes are stored. If you omit the package statement, the class names are put into the default /package, which has no name.

General form of defining a package;

package pkg;

package major;



HOW TO DEFINE A USER-DEFINED PACKAGE

```
package p1;
class c1
{
public void m1()
{
System.out.println("m1 of c1");
}
public static void main(string
args[])
{
c1 obj = new c1();
obj.m1();
}
}
```



ASSIGNMENT

- Write out 10 classes in pre-defined package
- Write on Recursive algorithm



EXCEPTION HANDLING

- **An exception (or exceptional event)** is a problem that arises during the execution of a program. When an Exception occurs the normal flow of the program is disrupted and the program/application terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.
- The exception handling in java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.
- Exception Handling is a mechanism to handle runtime errors such as ClassNotFound, IO, SQL, Remote etc.
- The core advantage of exception handling is to **maintain the normal flow of the application.** Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

1. Code 1;
2. Code 2;
3. Code 3;
4. Code 4; // exception occurs
5. Code 5;
6. Code 6;
7. Code 7;



EXCEPTION HANDLING

- The core advantage of exception handling is to **maintain the normal flow of the application.** Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

1. Code 1;
2. Code 2;
3. Code 3;
4. Code 4; // exception occurs
5. Code 5;
6. Code 6;
7. Code 7;

It is an unxecpted problem that happens while your program is running. For example;

- Trying to divide a number by zero
- Trying to access a file that doesn't exist
- Trying to access an element in an array that doesn't exist.

When such problems occur, java creates an **exception object** that contains information about the error.



TYPES OF EXCEPTION

There are mainly two types of exceptions which includes:

- Checked
- Unchecked exception
- Error; where Error is considered as unchecked expression
- **Checked Exception:** These are exceptions that are checked at compile-time. Examples include `IOException` and `SQLException`. They are problems that java forces you to handle before your program can run. The java complier checks at compile time whether you've handled these exceptions. It can be handled by using **try-catch block** to handle the exception and by declaring the exception using the **throws** keyword in the method signature. Example such: Trying to read a file that doesn't exsit (FileNotFoundException)
- **Unchecked Exception:** they are problems that occur due to logical errors in your code. The classes that extend RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time rather they are checked at runtime.



TYPES OF EXCEPTION

- Examples include: dividing a number by zero (ArithmeticException), Accessing an index in an array that doesn't exsit (ArrayIndexOutOfBoundsException). It can be handled using a **try-catch** block.
- **Error:** Error is irrecoverable i.e they are usually beyond your control and cannot be handled by your program. They often occur at the ystem level and are not meant to be caught or handled by your code. e.g. Running out of memory (OutOfMemoryError) , VirtualMachineError, AssertionError etc



SUMMARY OF EXCEPTION TYPES

TYPE	CHECKED/UNCHECKED	CAUSED BY	EXAMPLE
Checked Exception	Checked	External factors (files, databases, etc)	FileNotFoundException, SQLException
Unchecked Exception	Unchecked	Logical errors in code	NullPointerException, ArithmeticException
Errors	Unchecked	System-level failures	OutOfMemoryError, StackOverflowError



COMMON SCENARIOS WHERE EXCEPTIONS MAY OCCUR

There are given some scenarios where unchecked exceptions can occur. They are as follows:

- **Scenario where ArithmeticException occurs**

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

- **Scenario where NullPointerException occurs**

If we have null value in any variable, performing any operation by the variable occurs an NullPointerException.

```
String s=null;

System.out.println(s.length());//NullPointerException
```



COMMON SCENARIOS WHERE EXCEPTIONS MAY OCCUR

- **Scenario where NumberFormatException occurs**

The wrong formatting of any value, may occur NumberFormatException. Suppose I have a string variable that have characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";

int i=Integer.parseInt(s);//NumberFormatException
```

- **Scenario where ArrayIndexOutOfBoundsException occurs**

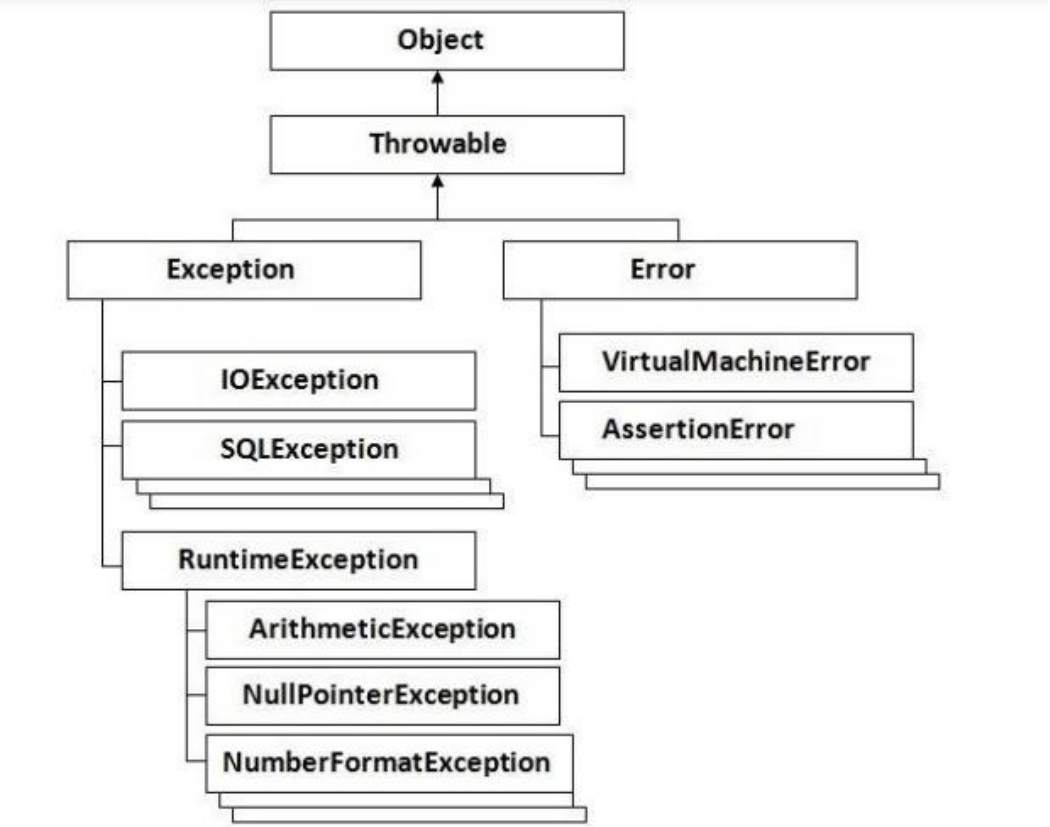
If you are inserting any value in the wrong index, it would result ArrayIndexOutOfBoundsException as shown below:

```
int a[]=new int[5];

a[10]=50; //ArrayIndexOutOfBoundsException
```



HIERARCHY OF JAVA EXCEPTION CLASSES



HIERARCHY OF JAVA EXCEPTION CLASSES

- In Java, exceptions are events that disrupt the normal flow of a program's instructions. The hierarchy of Java exception classes is rooted in the `Throwable` class, which is the superclass of all errors and exceptions in the Java language. Here's a detailed explanation of the hierarchy:

- Throwable: This is the root class for all exceptions and errors in Java. It provides a way to capture the state of the program when an exception or error occurs, including a stack trace and a message.

- Exception: This class and its subclasses represent conditions that a reasonable application might want to catch. Exceptions are divided into two main categories.

- Checked Exceptions: These are exceptions that are checked at compile-time. They must be either caught or declared to be thrown. Examples include `IOException` and `SQLException`.

- IOException: This is a general class of exceptions produced by failed or interrupted I/O operations.



HIERARCHY OF JAVA EXCEPTION CLASSES

- SQLException: This provides information on a database access error or other errors related to SQL operations.

- **Unchecked Exceptions (Runtime Exceptions):** These are exceptions that are not checked at compile-time. They are usually the result of programming bugs, such as logic errors or improper use of an API. Examples include `ArithmeticException`, `NullPointerException`, and `NumberFormatException`.

- **ArithmeticException:** This is thrown when an exceptional arithmetic condition has occurred, such as division by zero.

- **NullPointerException:** This is thrown when an application attempts to use `null` in a case where an object is required.



HIERARCHY OF JAVA EXCEPTION CLASSES

- NumberFormatException: This is thrown to indicate that the application has attempted to convert a string to one of the numeric types, but that the string does not have the appropriate format.

- **Error:** This class and its subclasses represent serious problems that a reasonable application should not try to catch. Errors are typically used to indicate issues with the Java Virtual Machine (JVM) itself or the environment in which the application is running. Examples include `VirtualMachineError` and `AssertionError`.

- VirtualMachineError: This indicates that the JVM is broken or has run out of resources necessary for it to continue operating.

- AssertionError: This is thrown to indicate that an assertion has failed.



JAVA EXCEPTION HANDLING

There are 5 keywords used in java exception handling which are try, catch, finally, throw and throws.

Java try-catch

Java try block is used to enclose the code that might throw an exception. It must be used within the method. Java try block must be followed by either catch or finally block

Syntax of java try-catch

```
try{
//code that may throw exception
}catch(Exception_class_Name ref){}
```

Syntax of try-finally block

```
try{
//code that may throw exception
}finally{}
```



JAVA CATCH BLOCK

Java catch block is used to handle the Exception. It must be used after the try block only. You can use multiple catch block with a single try.

Problem without exception handling

Let's try to understand the problem if we don't use try-catch block.

```
public class Testtrycatch1{
public static void main(String args[]){
int data=50/0;//may throw exception
System.out.println("rest of the code...");
}
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero



SOLUTION BY EXCEPTION HANDLING

Let's see the solution of above problem by java try-catch block.

```
public class Testtrycatch2{
public static void main(String args[]){
try{
int data=50/0;
}catch(ArithmeticException e){System.out.println(e);}
System.out.println("rest of the code...");
}
}
```

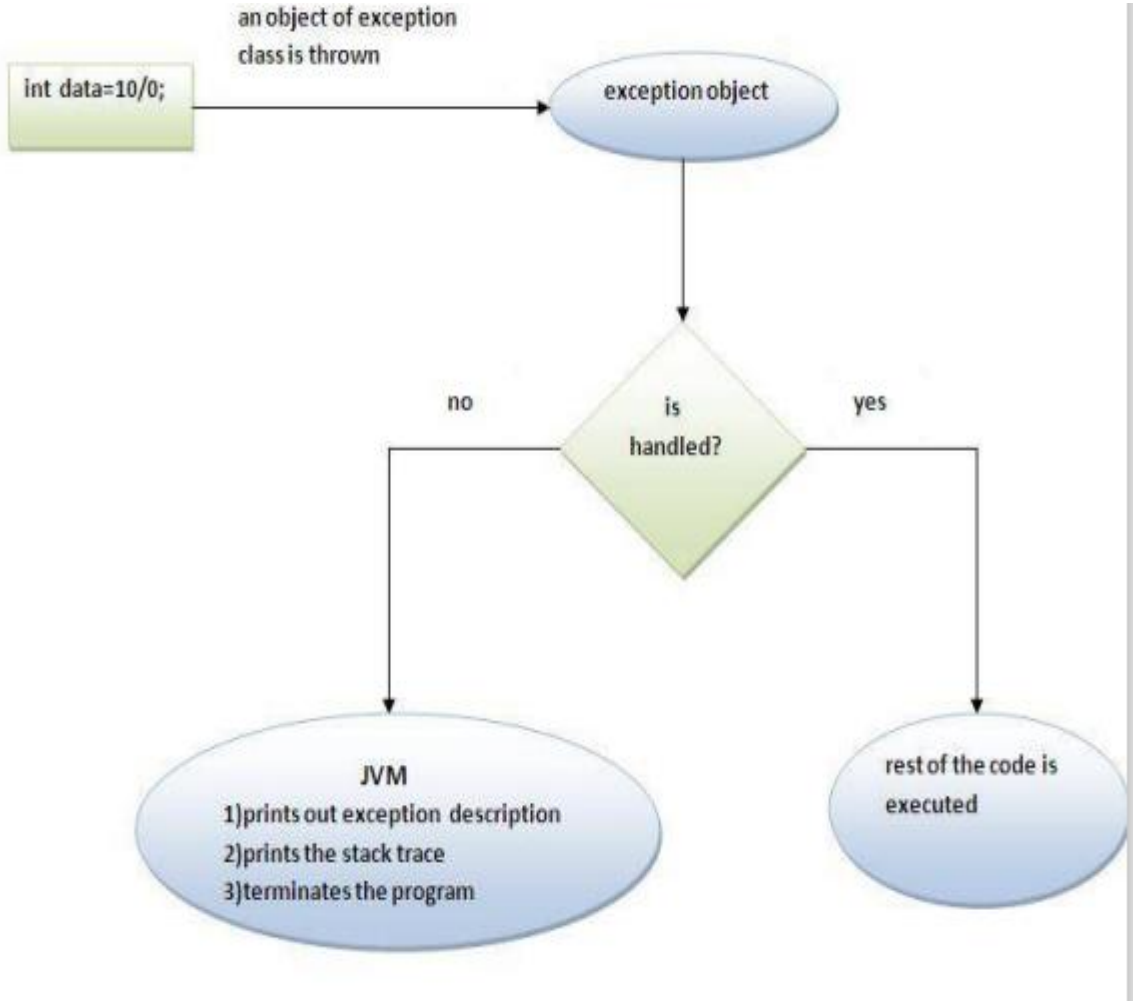
Output:

Exception in thread main java.lang.ArithmeticException:/ by zero

rest of the code...



INTERNAL WORKING OF JAVA TRY-CATCH BLOCK



INTERNAL WORKING OF JAVA TRY-CATCH BLOCK

The Java Virtual Machine firstly checks whether the exception is handled or not. If exception is not handled, Java Virtual Machine provides a default exception handler that performs the following tasks:

- Prints out exception description.
- Prints the stack trace (Hierarchy of methods where the exception occurred).
- Causes the program to terminate.

But if exception is handled by the application programmer, normal flow of the application is maintained i.e. rest of the code is executed



JAVA MULTI CATCH BLOCK

If you have to perform different tasks at the occurrence of different Exceptions, use java multi catch block.

Let's see a simple example of java multi-catch block.

```
public class TestMultipleCatchBlock{
public static void main(String args[]){
try{
int a[]=new int[5];
a[5]=30/0;
}
catch(ArithmeticException e){System.out.println("task1 is completed");}
catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}
catch(Exception e){System.out.println("common task completed");}
System.out.println("rest of the code...");
}
}
```

Output:task1 completed

rest of the code...



JAVA MULTI CATCH BLOCK

- Rule: At a time only one Exception is occured and at a time only one catch block is executed.
- Rule: All catch blocks must be ordered from most specific to most general i.e. catch for ArithmeticException must come before catch for Exception.

```
class TestMultipleCatchBlock1{
public static void main(String args[]){
try{
int a[]=new int[5];
a[5]=30/0;
}
catch(Exception e){System.out.println("common task completed");}
catch(ArithmeticException e){System.out.println("task1 is completed");}
catch(ArrayIndexOutOfBoundsException e){System.out.println("task 2 completed");}
System.out.println("rest of the code...");
}
```

Output:

Compile-time error



JAVA NESTED TRY BLOCK

- The try block within a try block is known as nested try block in java.

Why use nested try block?

- Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.



JAVA NESTED TRY BLOCK

Syntax

```
....
try
{
statement 1;
statement 2;
try
{
statement 1;
statement 2;
}
catch(Exception e)
{
}
}
catch(Exception e)
{
}
.....
```



JAVA NESTED TRY EXAMPLE

```
class Excep6{
public static void main(String args[]){
try{
try{
System.out.println("going to divide");
int b =39/0;
}catch(ArithmeticException e){System.out.println(e);}
try{
int a[]=new int[5];
a[5]=4;
}catch(ArrayIndexOutOfBoundsException e){System.out.println(e);}
System.out.println("other statement);
}catch(Exception e){System.out.println("handeled");}
System.out.println("normal flow..");
}
}
```

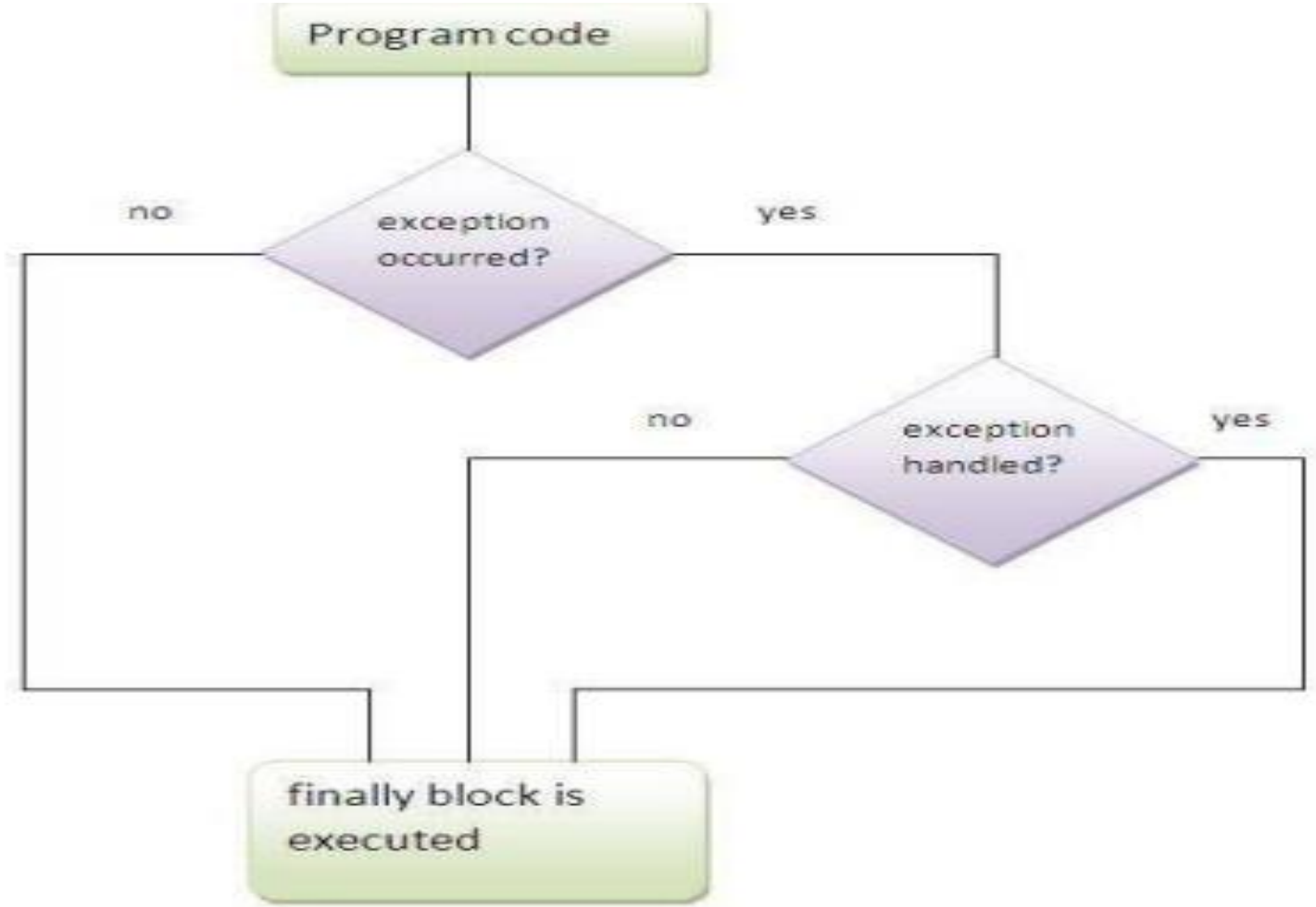


JAVA FINALLY BLOCK

- Java finally block is a block that is used to execute important code such as closing connection, stream etc.
- Java finally block is always executed whether exception is handled or not.
- Java finally block follows try or catch block



JAVA FINALLY BLOCK



USAGE OF JAVA FINALLY

- Let's see the different cases where java finally block can be used.

Case 1

Let's see the java finally example where exception doesn't occur

```
class TestFinallyBlock{
public static void main(String args[]){
try{
int data=25/5;
System.out.println(data);
}
catch(NullPointerException e){System.out.println(e);}
finally{System.out.println("finally block is always executed");}
System.out.println("rest of the code...");
}
}
```

Output:5

finally block is always executed

rest of the code...



USAGE OF JAVA FINALLY

Case 2 Let's see the java finally example where exception occurs and not handled.

```
class TestFinallyBlock1{
public static void main(String args[]){
try{
int data=25/0;
System.out.println(data);
}
catch(NullPointerException e){System.out.println(e);}
finally{System.out.println("finally block is always executed");}
System.out.println("rest of the code...");
}
}
```

Output: Exception in thread main java.lang.ArithmeticException:/ by zero
finally block is always executed
rest of the code...



- Rule: For each try block there can be zero or more catch blocks, but only one finally block.
- Note: The finally block will not be executed if program exits(either by calling System.exit() or by causing a fatal error that causes the process to abort).

